To do: Make a submission

Due: Sunday, 3 May 2026, 11:00 PM

Introduction

In this lab, you will:

- Set up a Python ML project environment with dependencies and testing.
- Create a Makefile to automate common tasks.
- Write a simple Python function and test it.
- Set up GitHub Actions for CI/CD to automate testing upon each push to GitHub
- Set up an Azure account and configure the Azure CLI.
- Deploy a Flask application to Azure App Services.
- Configure Azure DevOps to create a CI/CD pipeline that integrates with GitHub.
- Verify application functionality and troubleshoot deployment issues.

Instructions

Complete the steps below:

Section 1: Github CI/CD

Step 1: Project Setup and Virtual Environment

1. Create a new directory for your project and navigate into it:
   ▶ **Click to display code**

---

2. Make sure to check installation of python before setting up a Python virtual environment:
   ▶ **Click to display code**

---

3. Create a requirements.txt file for project dependencies:
   ▶ **Click to display code**

---

4. Install dependencies using pip:
   ▶ **Click to display code**

Step 2: Create a Simple Python Script

1. Create a Python file, main.py, with a basic function:
   ▶ **Click to display code**

---

2. Run main.py to verify the function works as expected:
   ▶ **Click to display code**

Step 3: Set Up Testing with Pytest

1. Create a test file test_main.py to test the add function:
   ▶ **Click to display code**

---

2. Run tests locally using Pytest:

?

▶ **Click to display code**

Step 4: Automate with a Makefile

1. Create a Makefile to automate setup, testing, and running the project:
   ▶ **Click to display code**

---

2. Test each Makefile command:
   - Install Dependencies:
     ▶ **Click to display code**

   - Run Tests:
     ▶ **Click to display code**

   - Run Program:
     ▶ **Click to display code**

Step 5: Set Up CI/CD with GitHub Actions

1. Initialize a Git repository and commit your changes:
   ▶ **Click to display code**

---

2. Create a .github/workflows directory:
   ▶ **Click to display code**

---

3. Create a GitHub Actions workflow file ci.yml:
   ▶ **Click to display code**

---

4. Push your code to GitHub (you may need to login to your GitHub account):
   ▶ **Click to display code**

---

5. Check GitHub Actions for CI results:
   - Go to your GitHub repository, navigate to the Actions tab, and verify that the workflow ran successfully.
     Expected Output: GitHub Actions workflow completes with successful test results:
     ```
     ✓ Checkout code
     ✓ Set up Python
     ✓ Install dependencies
     ✓ Run tests
     ```

<u>Section 2: Azure Deployment</u>

Step 1: Set Up the Environment

Install the Azure CLI if you haven't already:

▶ **Click to display code**

---

1. Log in to Azure from the command line:
   ▶ **Click to display code**

---

2. Create a Python virtual environment for your Flask project:
   ▶ **Click to display code**

---

3. Install Flask and create a simple application:
   ▶ **Click to display code**

Step 2: Develop the Flask Application

1. In your project directory, create a file named app.py with the following content:
   ▶ **Click to display code**

---

2. Test the application locally:
   ▶ **Click to display code**

   Verify that it runs by navigating to http://127.0.0.1:5000 in a browser.

Step 3: Deploy the Application to Azure App Services

1. Create a resource group:
   ▶ **Click to display code**

---

2. Deploy the Flask app to Azure App Services:
   ▶ **Click to display code**

   Replace <your-app-name> with a unique app name. The B1 SKU is a basic tier suitable for testing.

---

3. Verify the deployment by visiting https://<your-app-name>.azurewebsites.net.

Step 4: Configure Azure DevOps Pipeline for CI/CD

1. Create a GitHub repository and push your Flask application code to it.
2. In Azure DevOps, create a new project, and go to Pipelines to set up a new pipeline. Connect it to your GitHub repository.
3. Add the following Azure Pipeline configuration to your project root as azure-pipelines.yml:
   ▶ **Click to display code**

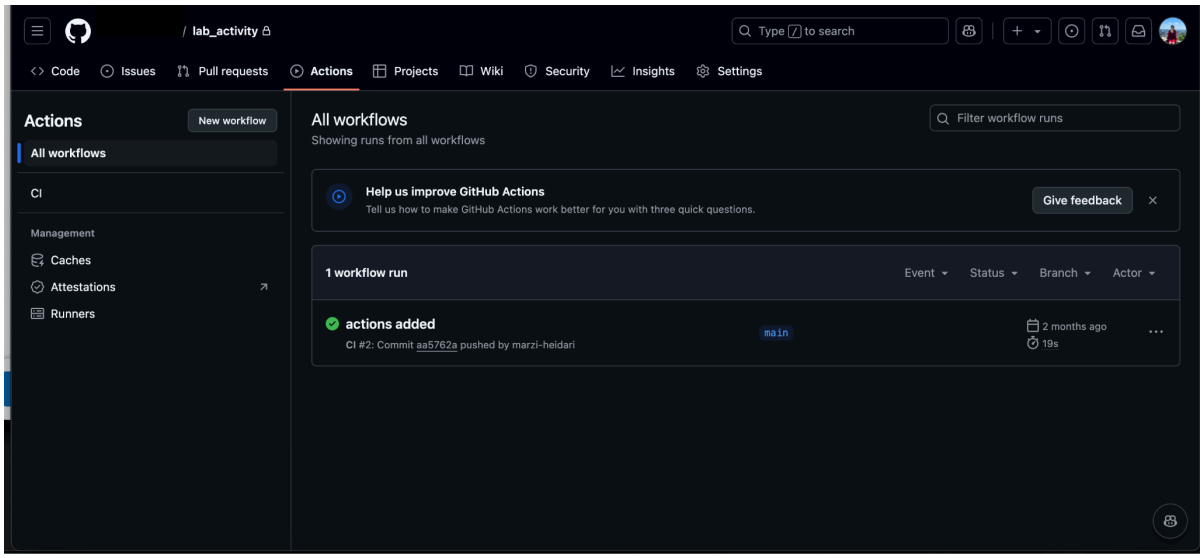
   Replace <your-service-connection-id> with the ID for your Azure subscription and <your-app-name> with your app's name.

Step 5: Verify CI/CD Deployment

1. Make a small change in app.py (e.g., change the message returned in home()).
2. Commit and push your changes to GitHub. This will trigger the pipeline.
   - To commit the changes run:
     ▶ **Click to display code**

   - To push the changes run:
     ▶ **Click to display code**

   - On GitHub, navigate to your repository and click on the "Actions" tab. You should see a new workflow run triggered by your recent push.



3. Verify that the pipeline runs successfully in Azure DevOps and that the updated application is redeployed to Azure App Services.
   - Run:
     ▶ **Click to display code**

   - For a specific pipeline run:
     ▶ **Click to display code**
   - status: Indicates the current status (inProgress, completed, etc.).
   - result: Shows the outcome (succeeded, failed, etc.).

Add submission

Submission status

Submission status	No submissions have been made yet
Grading status	Not graded
Time remaining	5 days 12 hours remaining